PYTHON MASTERY ROADMAP (UPDATED)

Ansh's practical learning plan | 3 November 2026 - 28 February 2027 | items marked **NEW** are additions to the original plan

Mission: Build strong Python fundamentals, independent problem-solving ability, and a foundation for Data Analysis / AI-ML. This plan prioritizes writing code daily, not just watching tutorials. This update adds SQL, early statistics, a retrieval-practice system, continuous GitHub commits and built-in buffer time.

1. Daily study commitment

Study mode	Time	How to use it
Minimum target	3-4 hours	The standard routine. Aim for this consistently, 6 days a week.
Strong day	5-6 hours	Add extra coding problems, debugging, and project work.
Maximum day	7-8 hours	Optional; use breaks. Do not force 8 hours every day or sacrifice sleep.

Recommended 4-hour daily routine

Block	Time
Concept + examples	45 min
Write code yourself (topic practice)	75 min
Problem solving (easy to medium)	60 min
Mini-project / debugging	45 min
Revision + notes + GitHub commit **NEW**	15 min

Weekly rhythm: 6 study days + 1 lighter review/rest day. On the review day, revise, redo 2-3 old problems, plan the next week, and do the **NEW** **weekly blank-file test** (rebuild a small program from scratch with no notes).

NEW **Extra daily habits:** 10 minutes of Anki flashcards for syntax and concepts you forget; commit to GitHub at least 3 times a week.

2. Month-by-month roadmap

NOVEMBER 3-30 | Python foundations & logic

- Setup: Python 3, VS Code, running scripts, input/output, comments.
- Variables, data types, type conversion, operators, strings and string methods.
- **NEW** **f-strings and string formatting.** Used in every program.
- Conditions: if / elif / else; logical and comparison operators.
- Loops: while, for, range, break, continue; nested loops basics.
- Collections: lists, tuples, sets, dictionaries; indexing, slicing, common methods.
- **NEW** **Mutability:** why lists change in place but strings don't; shallow vs deep copy.
- Functions: parameters, return, scope, default arguments; basic modular code.
- **NEW** **Basic algorithms by hand:** swapping, counting, min/max, reversing, palindromes, prime checks.
- Practice target: 35-45 small problems. Build a terminal calculator, number guessing game, unit converter, and simple quiz.

DECEMBER | Core Python & writing maintainable programs

- Functions deeper: positional/keyword arguments, *args and **kwargs (intro), recursion basics.
- Exception handling: try / except / else / finally; validate user input.
- Files: read/write text and CSV; with statement; paths.
- **NEW** **pathlib, datetime, random, collections (Counter, defaultdict), itertools basics.**
- Modules, imports, pip, virtual environments; use standard-library modules.

- OOP: classes, objects, __init__, instance attributes, methods, inheritance and encapsulation basics.
- NEW **Dunder methods:** __str__, __repr__, __eq__.
- Comprehensions, lambda, sorted/key, enumerate, zip; iterators and generators (intro).
- NEW **Type hints (light use) and code style:** PEP 8 plus a formatter such as black or ruff.
- Debugging, readable code, docstrings, Git/GitHub basics and Big-O intuition.
- Practice target: 30-40 problems. Projects: expense tracker (file-based), contact book, and a text-based OOP game (hangman or simple inventory).

JANUARY | Data handling & analysis

- NumPy: arrays, shapes, indexing, slicing, vectorized operations and basic statistics.
- NEW **Basic statistics alongside NumPy:** mean, median, standard deviation, distributions, correlation (20 min, a few times a week).
- Pandas: Series/DataFrame, read CSV/Excel, inspect data, select/filter/sort.
- Data cleaning: missing values, duplicates, data types, strings, dates and groupby.
- Analysis: aggregations, merge/join, pivot tables and summary statistics.
- NEW **SQL basics:** SQLite with sqlite3 (or SQL with pandas): SELECT, WHERE, ORDER BY, GROUP BY, JOIN. The biggest gap for a Data Analysis path.
- NEW Visualization: Matplotlib basics; charts, labels, titles and readable plots. **Add Seaborn** for fast exploratory charts.
- NEW **Jupyter/Colab habits:** markdown explanations, restart-and-run-all before sharing.
- Mini-projects: analyze a sales dataset and an expenses/student-performance dataset.
- Practice target: 2 complete notebooks with clear questions, cleaning, analysis and conclusions.

FEBRUARY | Automation, APIs & portfolio

- JSON and APIs: requests, GET requests, status codes, parsing JSON; handle errors.
- NEW **Web scraping basics** with BeautifulSoup (respect site terms). Optional; first to drop if behind.
- Automation: organize files, rename batches, work with folders and CSV/Excel reports.
- NEW **Command-line basics:** terminal navigation, argparse, Git branching.
- Testing basics: assertions and pytest introduction; logging and configuration basics.
- NEW Build one capstone project using Python and data. **Make it answer a real question** (e.g. "What factors affect X?").
- NEW **Simple Streamlit dashboard** to turn a notebook into a shareable app. Optional.
- GitHub: clean repositories, README, screenshots/plots, setup instructions and project explanations.
- Practice target: finish 3-5 presentable projects overall; revise weak Python fundamentals.

3. Week-by-week checkpoints

Dates	Main focus	Proof of learning
Nov 3-8	Setup, variables, types, input/output, operators, f-strings	Write 8-10 tiny programs without copying.
Nov 9-15	Strings, conditions, boolean logic	Build a grade calculator and simple menu program.
Nov 16-22	while / for loops, range, break / continue, basic algorithms	Number guessing game; solve loop patterns.
Nov 23-30	Lists, tuples, sets, dictionaries, mutability; functions intro	Contact list / quiz; explain each function aloud. **Month-end self-assessment.**
Dec 1-7	Functions deeper, scope, modules, standard library	Split a small program into functions.
Dec 8-14	Exceptions, validation, files, pathlib	File-based notes or expense tracker.
Dec 15-21	OOP foundations, dunder methods	Classes for a bank account or library; start the text-based OOP game.

Dates	Main focus	Proof of learning
Dec 22-31	**Catch-up and polish week** (no new topics): comprehensions, debugging, Git, style	Polish and upload 1-2 projects with READMEs. **Month-end self-assessment.**
Jan 1-7	NumPy basics + statistics habit starts	Notebook with array operations and summary stats.
Jan 8-14	Pandas basics and data loading	Explore a CSV dataset.
Jan 15-21	Cleaning, grouping, merging; SQL basics	Clean a messy dataset and document steps; run 10 SQL queries.
Jan 22-31	Visualization (Matplotlib + Seaborn) and analysis project; last 3 days flexible	Publish one complete analysis notebook. **Month-end self-assessment.**
Feb 1-7	JSON, APIs, requests (scraping optional)	Fetch and process data from a public API.
Feb 8-14	Automation, CSV/Excel, logging, command-line basics	Build a useful automation script.
Feb 15-21	Capstone project	Working project answering a real question, with a clear README.
Feb 22-28	Revision, testing, portfolio polish (Streamlit optional)	3-5 projects reviewed and presented on GitHub. **Final self-assessment.**

4. Retrieval and revision system NEW

- **Weekly blank-file test:** every review day, rebuild a small program from scratch with no notes.
- **Spaced repetition:** put forgotten syntax and concepts into Anki, 10 minutes a day.
- **Explain-back:** once a week, explain a concept in writing or aloud as if teaching a friend. Extend the function explanation habit to every topic.
- **Missed-problem sheet:** a simple spreadsheet with problem, date missed, and revisit dates (2 days and 1 week later).
- **Monthly self-assessment:** at each month-end, pick one unseen problem or small project and finish it in a set time without help. If you can't, spend the next week on that weakness before moving on.

5. Where to practice NEW

Goal	Platforms / resources
Beginner logic	HackerRank Python track, Exercism (mentor-style feedback), CodingBat
Medium problems	LeetCode Easy; a few Mediums from December onward
Data practice	Kaggle datasets and the free Kaggle Pandas and Data Visualization micro-courses
Datasets for projects	Pick ones from your own life or interests (cricket, college marks, personal spending); projects you care about get finished

6. Project ladder

Level	Project	Skills demonstrated
1	Terminal calculator with repeat menu and safe input	Conditions, loops, functions, exceptions
2	Number guessing game + quiz app	Loops, random, scoring, input validation
3	Expense tracker saved to CSV	Files, CSV, functions, data structures
4	Contact book / library manager	OOP, search, update, delete, persistence
4.5 **NEW**	Text-based OOP game (hangman or simple inventory/RPG)	OOP design, dunder methods, game state
5	Sales or personal-expense data analysis (add a few SQL queries)	Pandas, SQL, cleaning, aggregation, charts

Level	Project	Skills demonstrated
6	API data collector or automation tool	Requests, JSON, error handling, logging
7	Capstone data project answering a real question (optional Streamlit dashboard)	End-to-end problem solving, documentation, GitHub

NEW **README standard from project 1:** what it does, how to run it, one screenshot. Doing this from week one avoids a rush in February.

7. Rules and habits that make the plan work

- Type every example yourself. Do not just watch a tutorial and assume you have learned it.
- When stuck, try for 15-25 minutes: read the error, isolate the issue, test a small example, then ask for a hint.
- Keep an error log: error message, cause, fix, and what you learned.
- After learning a topic, solve problems without looking at notes; revisit missed problems after 2 days and 1 week.
- NEW Use AI as a tutor and reviewer, not as a code generator. **Rule: no AI help until you have a working or attempted solution.**
- NEW **Read other people's code:** 15 minutes a week on a small, well-written GitHub repo.
- NEW **Practice reading official docs** (Python, pandas) and error messages, not just tutorials.
- NEW **Cut order if you fall behind:** protect loops, functions and data structures first. Drop Streamlit, then web scraping, first.
- Do not measure mastery only by hours. Measure it by what you can build and explain independently.
- Three to four focused hours daily is a strong, sustainable minimum. Eight hours is optional. Protect sleep and your rest day.

8. March 2027 | Revision & interview preparation

March is a consolidation month, not a reason to rush into advanced AI/ML without fundamentals. Revise core Python, solve mixed problems, explain your projects, improve GitHub READMEs, and practice basic coding interviews. Then begin a structured path through statistics, linear algebra basics, NumPy/Pandas fluency, and introductory machine learning. Because you started a light statistics habit in January, probability and intro ML will build on something you already know.

Important expectation

Four months of consistent work can build a strong foundation and a useful beginner portfolio, but nobody can honestly guarantee complete mastery of Python or job readiness in a fixed number of months. Keep practicing beyond this roadmap. The goal is independent problem-solving and steady progress toward Data Science / AI-ML.

Start today: 3 November 2026. Create a Python file, revise input(), type conversion and operators, then write a small program yourself.